UNIT-2
Important Questions

1(a) Define Register Transfer language (RTL). Explain the purpose of RTL in digital system design.

Ans **Register Transfer language:-** RTL is a symbolic notation used to describe micro operations, data transfers and control operations that occur between registers in a digital system. It specifies how data is moved, processed and controlled among registers during executions of instructions.

** RTL represents register operations and data transfers using symbols instead of long descriptive statements.

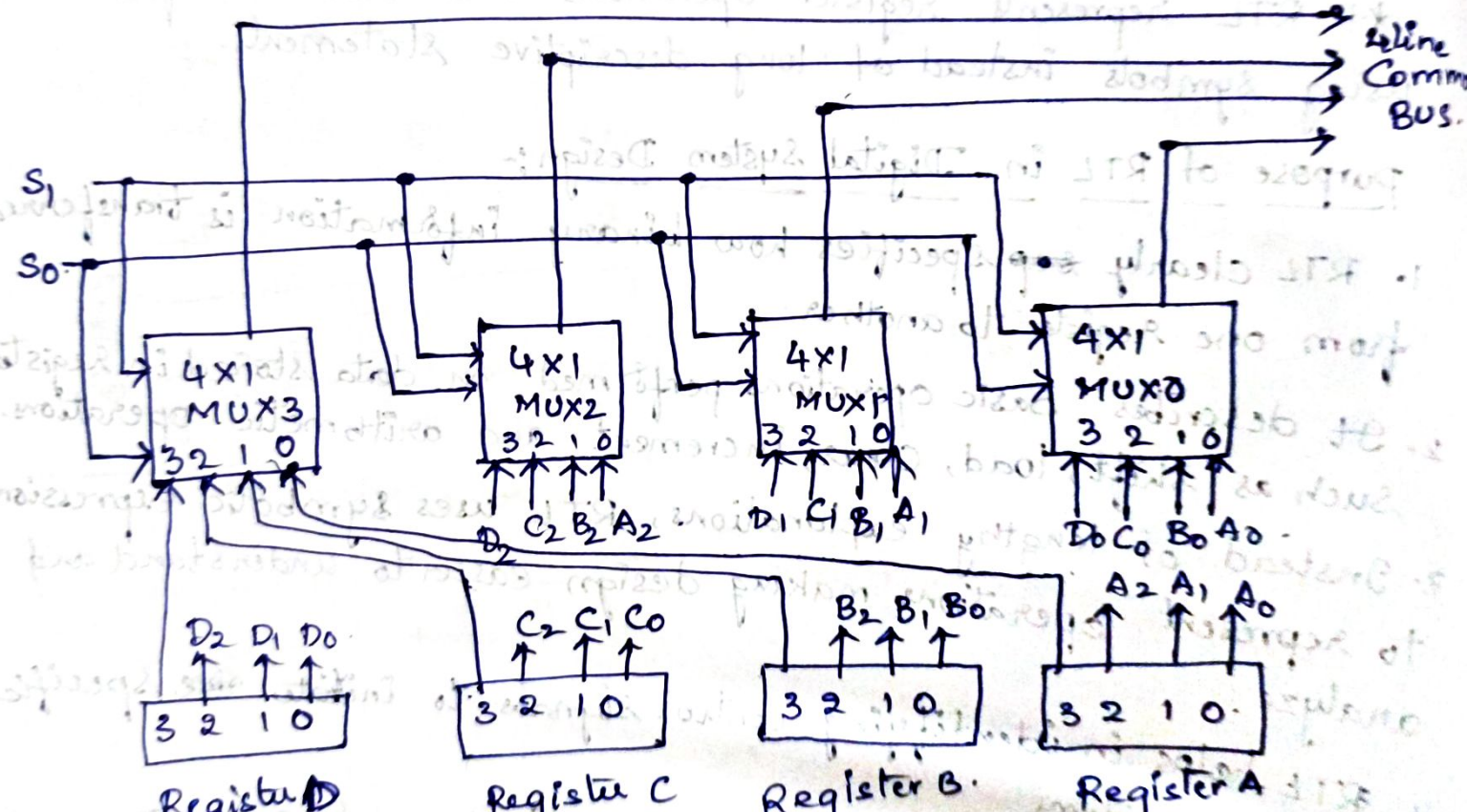
Purpose of RTL in Digital System Design:-

1. RTL clearly ~~sep~~ specifies how binary information is transferred from one register to another.
2. It describes basic operations performed on data stored in registers such as shift, load, clear, increment and arithmetic operations.
3. Instead of lengthy explanations, RTL uses symbolic expressions to represent operations making design easier to understand and analyze.
4. RTL helps in identifying control signals to initiate ~~use~~ specific micro operations
5. It is used to describe internal organization of computers and digital system including the sequence of operations performed on stored data
6. RTL provides a clear and concise way to document register operations ~~which is useful during~~

(b) Describe the Common Bus System using multiplexers with help of neat diagram. Explain how register selection is performed in bus system.

Ans: A Common Bus is used in digital computers to transfer Binary information between registers efficiently. Instead of connecting every register with seperate wires, a shared Communication path called a bus is used.

→ A Multiplexer Based Bus System is used to transfer data to destination registe. MUX selects one register at a time and place its contents on the Bus.



* Each MUX Corresponds to one bus line, for 4 bit bus, four MUX are required. Each MUX selects one bit from one register.

→ Assume 4 Registers A, B, C, D having 4 bit each, A 4X1 MUX is used for each bus line. The inputs of each MUX come from Corresponding bits of all registers. The o/ps of MUX from

the common bus lines. The selection lines choose which register places its data on the bus.

→ The MUX are controlled by two selection line S_1 and S_0. These lines determine which register is connected to bus

S_1	S_0	Register selected
0	0	A
0	1	B
1	0	C
1	1	D

Ex: If $S_1S_0 = 00$ Bus recieves data from Register A

$S_1S_0 = 01$ Bus recieves data from Register B

$S_1S_0 = 10$ Bus recieves data from Register C

$S_1S_0 = 11$ " " " Register D.

The selected register places its contents onto the common bus, and the destination register can load data using its load control signal

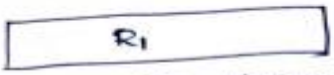
2a) Explain Register transfer with suitable notation and describe how control functions are used in register Transfer function.

Register Transfer:-

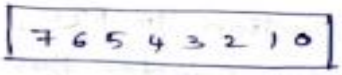
Registers in a digital system are named by capital letters

Ex: MAR : Memory Address Register
PC : Program Counter
IR : Instruction Register
R_1, R_2 → General purpose Registers.

Block Diagram of Registers:-

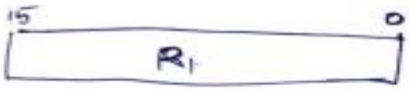


(or)

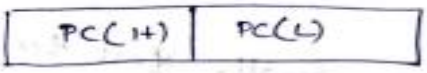


Usually Register shown in Rectangular box with Capital letter name inside.

Bits are numbered from right (LSB) to left most (MSB)



Register is divided into two parts
16 bit PC lower byte (L) → bits (0-7)
High byte PC(H) → bits (8-15)



Register transfer Concept :-

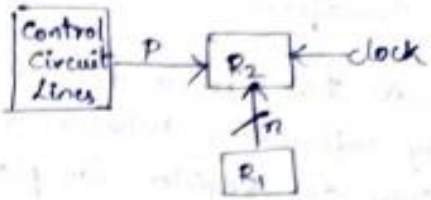
* Register transfer means moving binary information from one register to another.

Example Code = $R_2 \leftarrow R_1$ (This is Called Register transfe
Destination Source

Meaning :- * Content of R_1 is Copied into R_2.
* R_1 Value remains unchange.
* R_2 gets new Content.

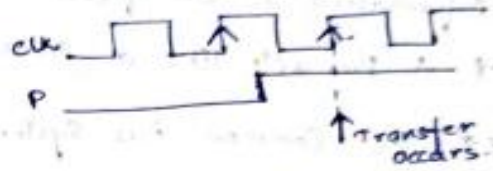
Control function in Register transform :-

* To occur register transfer
1) the outlines of source must connect to input lines of destination.
2) Transfer occurs on clock edge.



Block Diagram

Example Code :- If (P=1) then $R_2 \leftarrow R_1$ (OR)
in Compact RTL form
$P: R_2 \leftarrow R_1$



Timing diagram

* Contents of R_1 and R_2 are exchanged. happen in one clock pulse.

Basic symbols used in RTL :-

Symbol	Meaning	Example
Letters	Register name	R_1, MAR.
( )	Part of Register	$R_2(0\text{-}7)$
$\leftarrow$	Transfer of data	$R_2 \leftarrow R_1$
,	Simultaneous operation	$R_1 \leftarrow R_2, R_2 \leftarrow R_3$
:	Control Conditions	$P: R_2 \leftarrow R_1$

$R_1 \leftarrow$ BUS

(b) Describe Common bus system using three state buffers with neat diagram

BUS using Three state Buffers :-

→ Three state Buffers Consists of three states

0

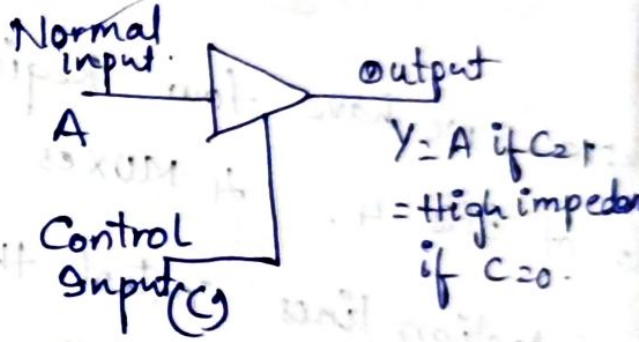
1

Z (High impedence).

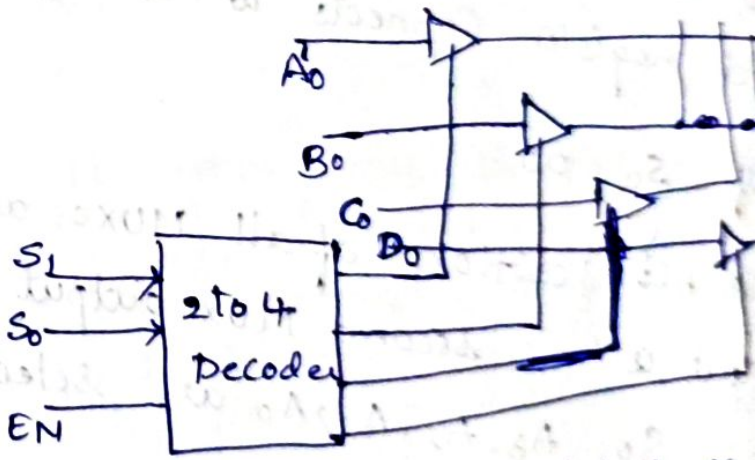
↳ (acts like open ckt)

→ Each register Connects to bus through buffer, only one buffer enabled at a time, Others remain in high impedence

→ Only one butter is active at a time that is enabled using decoder.



→ The diagram shows how a common bus line (for one bit) is implemented using three state buffers and a decoder. Instead of MUX.



→ Apply S_1 & S_0 and EN = 1

Decoder activates exactly one o/p line

→ The active decoder o/p enables corresponding three state buffer

→ The enabled buffer passes its input ($A_0/B_0/C_0/D_0$) to the bus. The other buffers remain in High impedence state (z).

→ To Construct a Common bus for four registers of n-bits each using three state Buffers, we need n circuits with 4 buffers

3(a) Explain memory read and memory write operations with appropriate register transfer statements.

Ans **Memory Transfer :**

* The Transfer of information from a memory word to the outside environment is called a "read operation".
* The Transfer of new information to be stored into the memory is called "write Operation".

→ Memory word will be symbolized by letter 'M'. The particular memory word among the many available is selected by the memory address during transfer. It is necessary to specify address of M when writing memory transfer operations. This will be done by enclosing the address in square brackets following the letter M.

→ Consider a memory unit that recieves the address from a register, called the address register, symbolized by AR. The data are transferred to another register, called the data register, symbolized by DR. The read operation can be stated as follows.

Read: $DR \leftarrow M[AR]$.

This causes a transfer of information into DR from the memory word M selected by the address in AR.

→ The write operation transfers the content of a data register to a memory word M selected by the address. Assume that the input data are in register R_1

$$R_3 \leftarrow R_1 + \overline{R_2} + 1$$

$\overline{R_2}$ is the symbol for 1's Complement of R_2. Adding 1 to 1's Complement produces 2's Complement.

(b) List and Explain the arithmetic microoperations. Describe the hardware implementation of a binary adder.

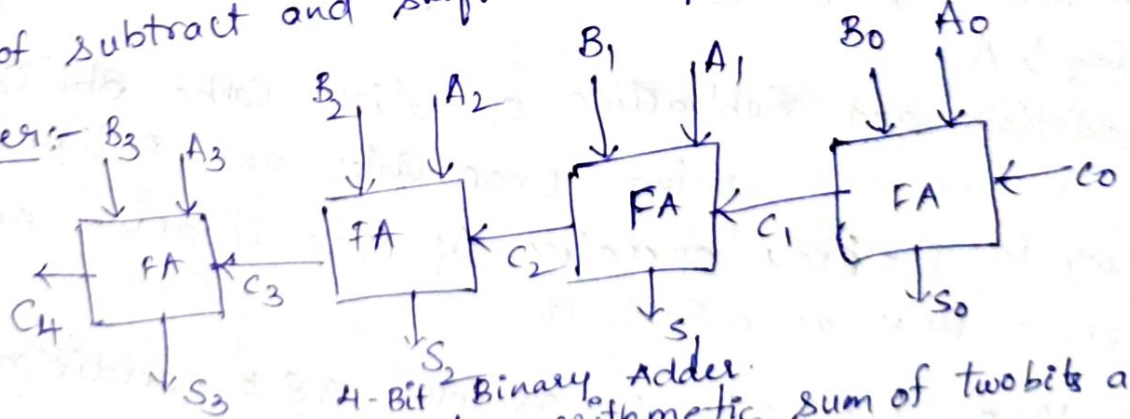
Arithmetic Micro operations:-

Symbolic designation	Description
$R_3 \leftarrow R_1 + R_2$	Contents of R_1 plus R_2 transferred to R_3
$R_3 \leftarrow R_1 - R_2$	Contents of R_1 minus R_2 transferred to R_3
$R_2 \leftarrow \overline{R_2}$	Complements of Contents of R_2 (1's complement)
$R_2 \leftarrow \overline{R_2} + 1$	2's Complement of Contents of R_2 (negation)
$R_3 \leftarrow R_1 + \overline{R_2} + 1$	R_1 plus 2's comp of R_2 (subtraction)
$R_1 \leftarrow R_1 + 1$	increment Contents of R_1 by one
$R_1 \leftarrow R_1 - 1$	decrement Contents of R_1 by one

→ Arithmatic operations multiply and divide are not included in basic set of micro operations.

→ In most computers, multiplication operation is implemented with a sequence of add and shift micro operations, Division is implemented by sequence of subtract and shift micro operations.

Binary Adder:-



4-Bit Binary Adder

→ A Digital circuit that forms the arithmetic sum of two bits and a previous Carry is called FULL ADDER, The digital ckt that generates arithmetic sum of two Binary numbers of any length is called a Binary adder.

→ Binary adder is constructed with FA ckts connected in cascade with o/p Carry of the next FA

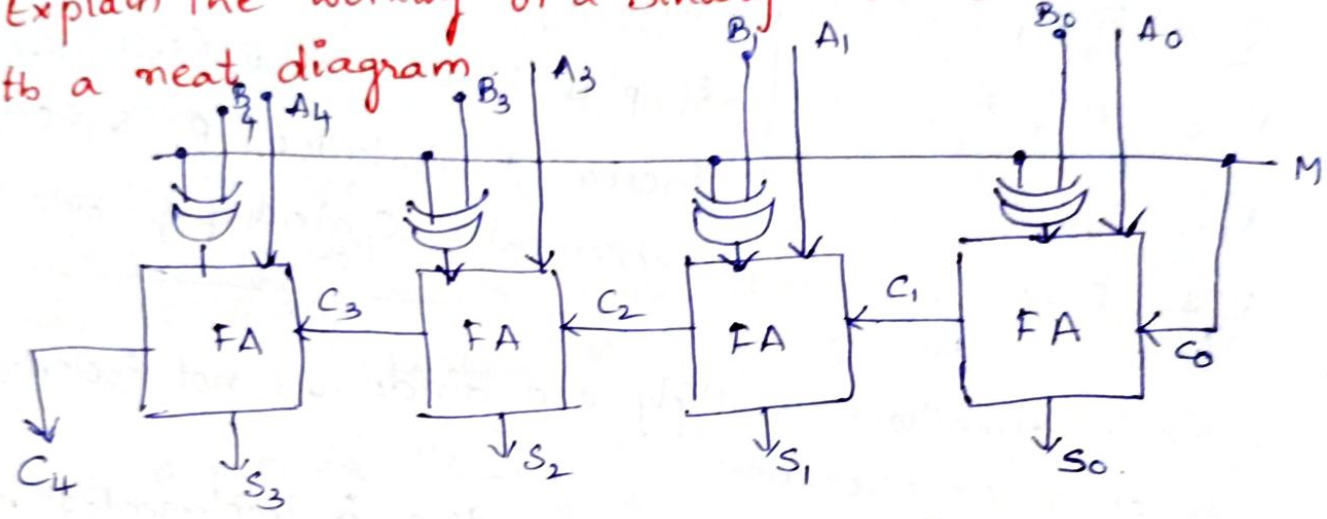
→ The augend bits of A and addend bits of B are designated by subscript numbers from right to left. The Carries are connected in a chain through FULL Adders.

A n-bit Binary adder requires n FULL Adders.

→ The n data bits of A inputs come to Register R_1, B inputs come to Register R_2, Sum can be transferred to a third register & to one of the source registers (R_1 & R_2) replacing its previous Content

4(a) Explain the working of a Binary adder-Subtractor ckt with a neat diagram

Ans



4-Bit Binary Adder-Subtractor Ckt

→ Subtraction A-B can be done by taking 2's Comp of B and adding to A.

→ Addition and Subtraction operations can be Combined into one Common ckt using EXOR Gates and FULL Adders.

→ Mode M Controls operation, If M=0 it acts as Adder, M=1, it acts as a Subtractor

→ If M=0 $B \oplus 0 = B$, FA recieves A & B, addition operation is done A+B

→ If M=1 $B \oplus 1 = \bar{B}$, FA recieves A; $C_0 = 1$, B inputs are all Complemented and 1 is added through input Carry

$S = A + \bar{B} + 1 = A + 2$'s Comp of $B = A - B$

~~5(a) Explain~~

~~4(b) Describe logic micro operations. Distinguish between~~

4(b) Describe Logic Micro Operations. Explain any four logic micro operations and with examples

→ Logic micro operations that perform bit-by-bit logical manipulation on the data stored in registers.

→ These operations treat each bit of the register as an independent binary variable (0 & 1). Logical operations are applied simultaneously to corresponding bits of two registers, and result is stored in destination register.

For example:- $P: R_1 \leftarrow R_1 \oplus R_2$.

→ P is control signal, P=1, Contents of R_1 replaced by bit by bit XOR result of R_1 and R_2

If $R_1 = 1010$

$R_2 = 1100$ EXOR of each bit and stored again in R_1

$R_1 = 0110$

List of Logic Micro operations:-

There are 16 different Micro operations, that can be performed.

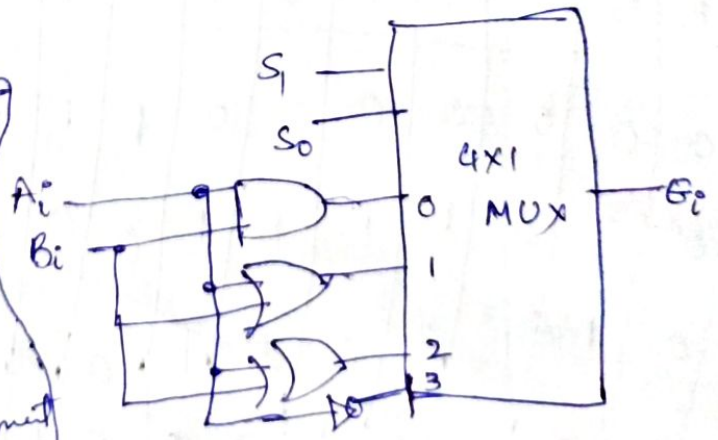
Truth table for 16 functions of two variables.

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

Boolean function	Micro operation	Name
$F_0 = 0$	$F \leftarrow 0$	Clear
$F_1 = xy$	$F \leftarrow A \wedge B$	AND
$F_2 = xy'$	$F \leftarrow A \wedge \bar{B}$	
$F_3 = x$	$F \leftarrow A$	Transfer A
$F_4 = x'y$	$F \leftarrow \bar{A} \wedge B$	
$F_5 = y$	$F \leftarrow B$	Transfer B
$F_6 = x \oplus y$	$F \leftarrow A \oplus B$	Exclusive OR
$F_7 = x + y$	$F \leftarrow A \vee B$	OR
$F_8 = (x+y)'$	$F \leftarrow \overline{A \vee B}$	NOR
$F_9 = (x \oplus y)'$	$F \leftarrow \overline{A \oplus B}$	EXNOR
$F_{10} = y'$	$F \leftarrow \bar{B}$	Complement of B
$F_{11} = x + y'$	$F \leftarrow A \vee \bar{B}$	
$F_{12} = x'$	$F \leftarrow \bar{A}$	Complement A
$F_{13} = x' + y$	$F \leftarrow \bar{A} \vee B$	
$F_{14} = (xy)'$	$F \leftarrow \overline{A \wedge B}$	NAND
$F_{15} = 1$	$F \leftarrow$ all 1's	Set to all 1's

Some Applications:

S_1	S_0	output	operation
0	0	$E = A \wedge B$	AND
0	1	$E = A \vee B$	OR
1	0	$E = A \oplus B$	EXOR
1	1	$E = \bar{A}$	Complement



① AND operation

A = 1010 B = 1100

F ← A ∧ B

A = 1010
B = 1100
AND = 1000 (AND of each bit)

② OR operation

A = 1010
B = 1100

A = 1010
B = 1100
A OR B = 1110 (OR of each bit)

F ← A ∨ B

③ EXOR operation

A = 1010
B = 1100
0110

A = 1010
B = 1100
A EXOR B = 0110 (EXOR of each bit)

F ← A ⊕ B

④ Complement operation :- complement each bit

A = 1010

F ← $\bar{A}$

$\bar{A}$ = 0101

5(a) Explain shift micro operations. Distinguish between logical, Circular and arithmetic shifts.

Ans → Shift Micro operations are used for serial transfer of data in registers.

→ While shifting, the bits move from one flip flop position to another.

→ Shifting is of three types

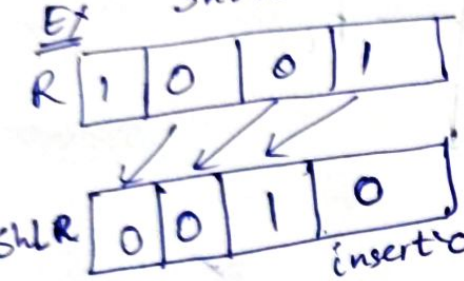
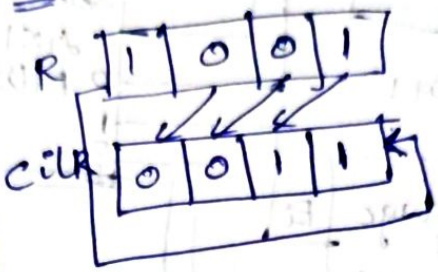
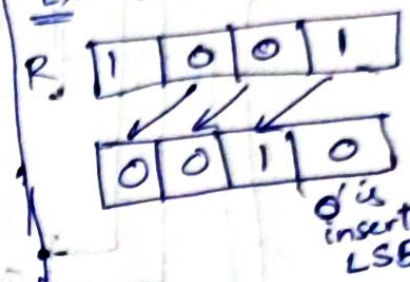
1. Logical shift
2. Circular shift
3. Arithmetic shift

Shift Micro operations

Symbolic designation	Description
R ← shl R	Shift left register R.
R ← shr R	Shift right register R.
R ← cil R	Circular shift left register R.
R ← cir R	Circular shift right register R.
R ← ashl R	Arithmetic shift left R
R ← ashr R	Arithmetic shift right R.

Differences:-

Logical Shift	Circular shift	Arithmetic shift
1. A Logical shift transfers 0 through serial input. In this shift all bits move one position to left or right and empty position is filled with '0'. 2. Symbols: shl → logical shift left shr → logical shift right $R_1 \leftarrow shl\ R_1$ $R_2 \leftarrow shr\ R_2$. 3. In logical shift left all bits move one position left and 0 enters in right most position	1. Also called rotate operation, Circulates the bits of the register around the two ends without losing any information. 2. Symbols: cil → Circular Shift left cir → Circular Shift right $R_1 \leftarrow cil\ R_1$ $R_2 \leftarrow cir\ R_2$ 3. In Circular shift left bit shift left of left end of register and re enters right end no bit is lost	1. It is used for signed binary number. The left most bit represent sign bit remaining are magnitude of number. 2. Symbols. ashr → Arithmetic shift Right ashl → Arimetic shift left 3. Arithmetic left shift, all bits shifts left '0' is inserted to left

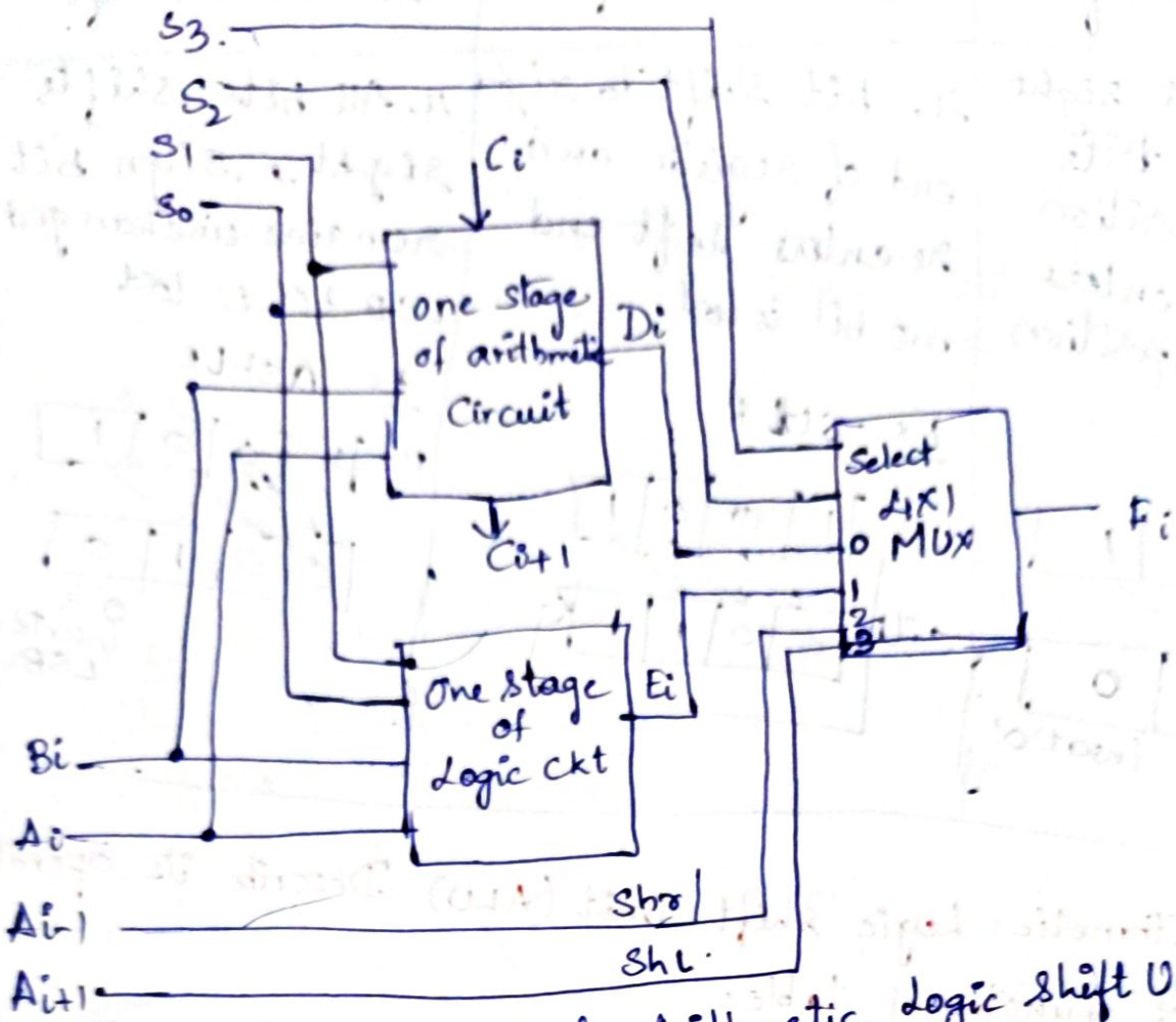
logical shift	Circular shift	Arithmetic shift
4. In logical right shift, all bits move one position to right, '0' enters to left most position Ex shlR  	4. bit shift to right end of register and re enters left end no bit is lost Ex. cilR  	4. All bits shifts right, sign bit remains unchanged. LSB bit is lost. Ex ashlR 

(b) Explain Arithmetic Logic shift Unit (ALU) Describe its operation with help of functional table.

→ Instead of having individual registers performing arithmetic, logic and shift micro operations, directly registers are connected to a common unit called Arithmetic Logic Unit (ALU).

→ ALU is a combinational circuit, so the entire register transfer operation from the source registers through the ALU to the destination register can be completed during one clock pulse.

→ Arithmetic, logic and shift operations can be combined into a single unit called ALU. (Arithmetic Logic shift Unit).



One Stage of Arithmetic Logic Shift Unit

→ 4X1 MUX at the output selects the required result, Controlled by inputs S_3, S_2, S_1 and S_0.

→ Arithmetic ckt produces output D_i

Logic ckt produces output E_i.

A_{i-1} → shift right operations

A_{i+1} → shift left operations.

C_i → Carry i/p.

, C_{i+1} → Output Carry.

The circuit provides.

* 8 Arithmetic operations. [when $S_3S_2 = 00$]
* 4 Logic operations. [when $S_3S_2 = 01$]
* 2 shift operations. [when $S_3S_2 = 10$ & 11].

operation select					operation	Function
S_3	S_2	S_1	S_0	Cin		
0	0	0	0	0	$F = A$	Transfer A.
0	0	0	0	1	$F = A + 1$	Increment A.
0	0	0	1	0	$F = A + B$	Addition.
0	0	0	1	1	$F = A + B + 1$	Add with Carry.
0	0	1	0	0	$F = A + \bar{B}$	Subtract with borrow
0	0	1	0	1	$F = A + \bar{B} + 1$	Subtraction.
0	0	1	1	0	$F = A - 1$	Decrement A
0	0	1	1	1	$F = A$	Transfer A.
0	1	0	0	X	$F = A \wedge B$	AND
0	1	0	1	X	$F = A \vee B$	OR
0	1	1	0	X	$F = A \oplus B$	EXOR.
0	1	1	1	X	$F = \bar{A}$	Complement of A.
1	0	X	X	X	$F = Shr\ A$	Shift right A into F
1	1	X	X	X	$F = Shl\ A$	Shift left A into F